

MNIST Handwritten Digit Classification with Convolutional Neural Networks

AI 100 - Midterm Project Report

Kreesh Patel

GitHub: github.com/kreeshpatel/ai100-mnist-cnn-classification

1. Problem Definition and Dataset Curation

The goal of this project is to build a deep learning model that can accurately classify handwritten digits (0 through 9) from grayscale images. Handwritten digit recognition is a foundational problem in computer vision and serves as an excellent benchmark for evaluating neural network architectures. It has real-world applications in postal mail sorting, bank check processing, and digitizing handwritten documents.

Dataset: MNIST

The MNIST (Modified National Institute of Standards and Technology) dataset is one of the most widely used benchmarks in machine learning. It was originally compiled by Yann LeCun and colleagues and contains handwritten digits collected from Census Bureau employees and high school students. The dataset is well-balanced across all ten digit classes, making it ideal for training and evaluating classification models.

Training set: 60,000 images

Test set: 10,000 images

Image size: 28 x 28 pixels, single-channel grayscale

Classes: 10 (digits 0-9)

Source: torchvision.datasets.MNIST (auto-downloaded)

Preprocessing

Each image is converted to a PyTorch tensor and normalized using the dataset's global mean (0.1307) and standard deviation (0.3081). This normalization centers pixel values around zero and scales them to roughly unit variance, which helps the network converge faster and more stably during training. No data augmentation was applied, as the standard MNIST benchmark does not typically require it to achieve high accuracy.

2. Deep Learning Model

The model is a Convolutional Neural Network (CNN) implemented in PyTorch. CNNs are the standard architecture for image classification tasks because their convolutional layers can automatically learn spatial features such as edges, curves, and shapes directly from raw pixel data, without requiring manual feature engineering.

Why a CNN?

Traditional fully connected networks treat each pixel independently and ignore the spatial structure of images. CNNs, on the other hand, use shared convolutional filters that slide across the image to detect local patterns. This parameter sharing makes CNNs more efficient and effective for image tasks. Pooling layers further reduce spatial dimensions while preserving the most important features, providing some degree of translation invariance.

Architecture (MNISTNet)

```

Input (1 x 28 x 28)
-> Conv2d(1, 32, kernel=3, padding=1) -> ReLU -> MaxPool(2)    [32x14x14]
-> Conv2d(32, 64, kernel=3, padding=1) -> ReLU -> MaxPool(2)  [64x7x7]
-> Flatten                                                    [3136]
-> Linear(3136, 128) -> ReLU
-> Linear(128, 10)                                           [10 logits]

```

The first convolutional layer extracts 32 low-level feature maps that capture basic patterns like edges and simple curves. The second convolutional layer builds 64 higher-level feature maps from those, capturing more complex shapes and digit components. Each convolutional layer is followed by a ReLU activation function for non-linearity and a 2x2 max-pooling layer that halves the spatial dimensions. The output is flattened into a 3,136-dimensional vector and passed through two fully connected layers, ultimately producing 10 raw logits (one per digit class). The total number of trainable parameters is approximately 206,000.

Training Configuration

Loss function: CrossEntropyLoss (combines LogSoftmax + NLLLoss)

Optimizer: Adam (lr = 0.001)

Batch size: 256

Epochs: 3

Random seed: 42 (for reproducibility)

Device: CPU

Adam was chosen as the optimizer because it adapts learning rates per parameter using estimates of first and second moments of the gradients, which generally leads to faster convergence than standard SGD on this type of task. CrossEntropyLoss is the standard choice for multi-class classification problems, as it combines a softmax operation with the negative log-likelihood loss in a numerically stable way.

3. Results

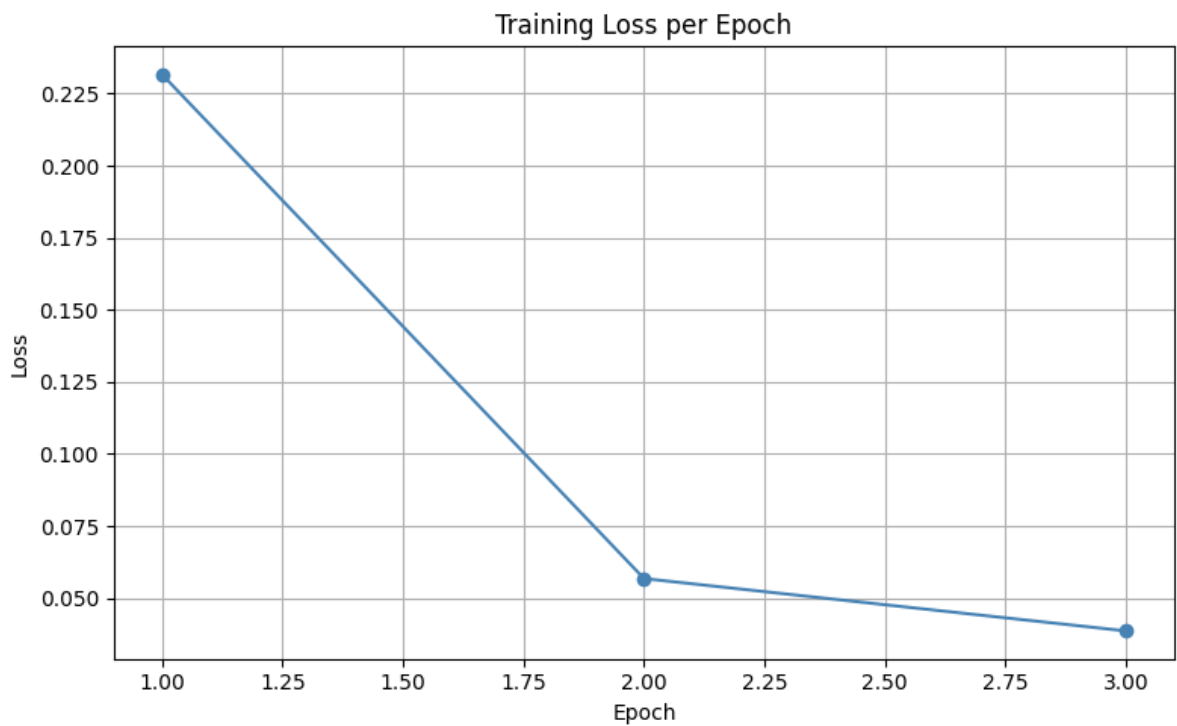
Training Performance

Epoch	Training Loss	Training Accuracy
1	0.2317	93.32%
2	0.0568	98.29%
3	0.0386	98.80%

Final Test Accuracy: 98.84%

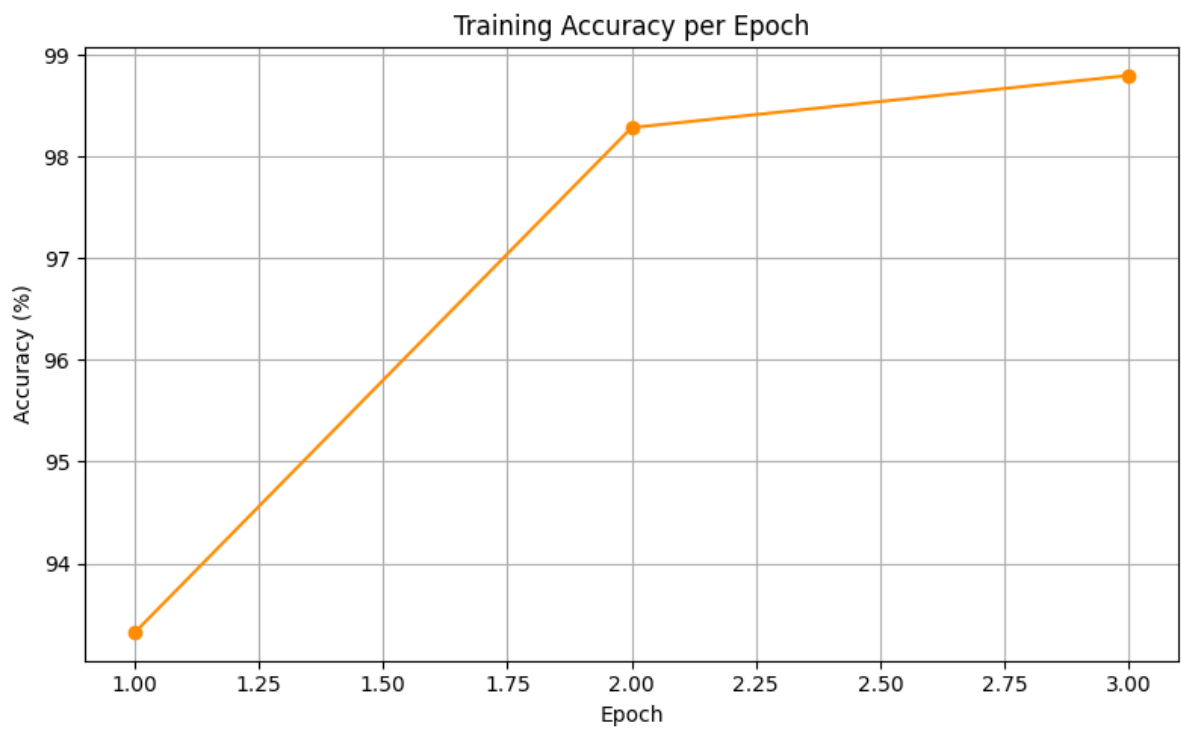
The model converges rapidly, achieving over 93% training accuracy after just the first epoch. By epoch 3, training accuracy reaches 98.80% and the test accuracy is 98.84%, demonstrating strong generalization with minimal overfitting. The close match between training and test accuracy indicates that the model learned genuine digit features rather than memorizing the training data.

Training Loss Curve



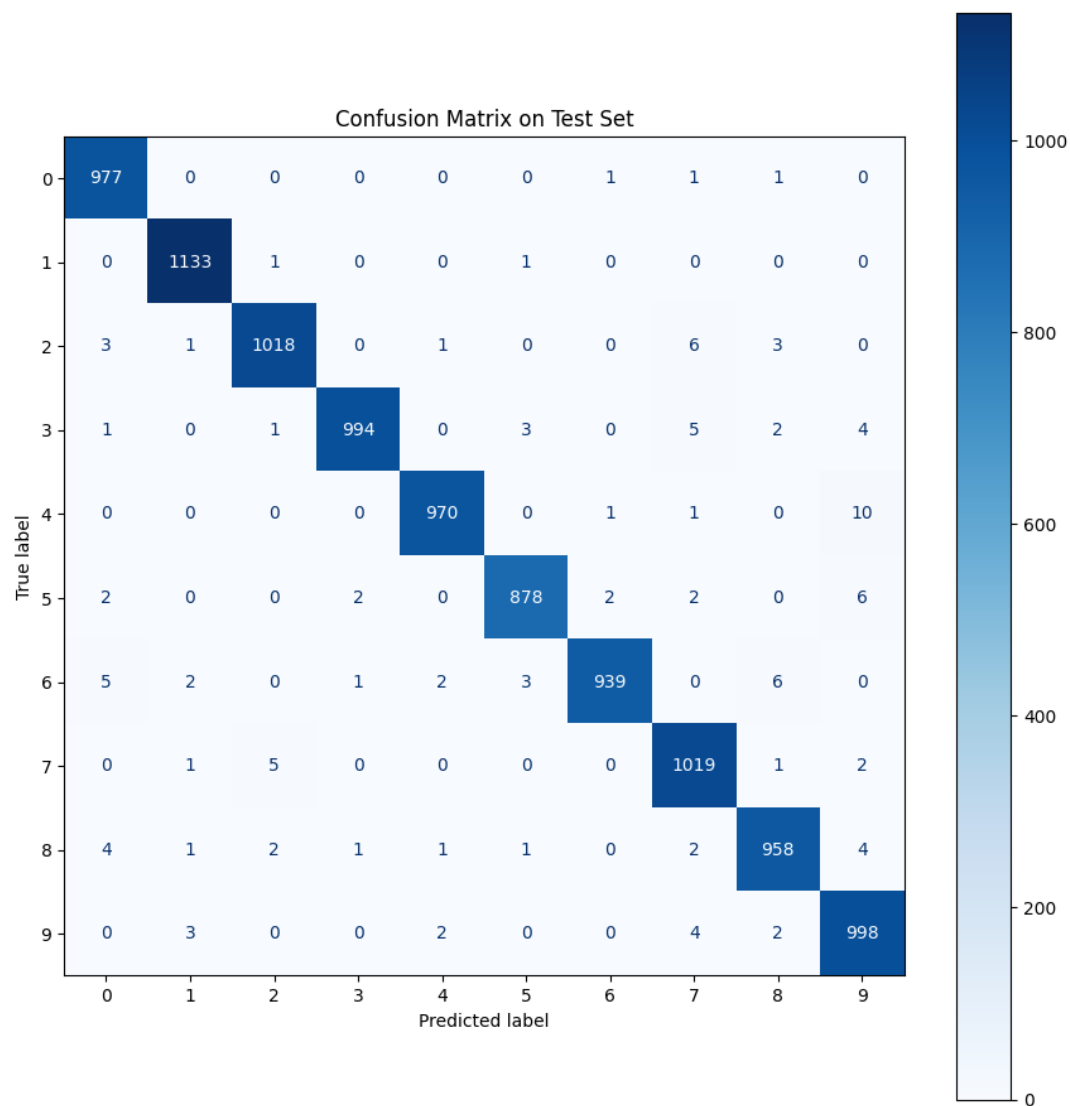
The loss decreases sharply after the first epoch and continues to decline steadily, indicating effective learning without any signs of divergence or instability.

Training Accuracy Curve



Training accuracy climbs quickly to above 98% by epoch 2 and reaches 98.80% by epoch 3, showing that the model has effectively learned the digit patterns and converged.

Confusion Matrix (Test Set)



The confusion matrix shows the model classifies nearly all test digits correctly. The diagonal is strongly dominant with very few off-diagonal misclassifications. The most common errors involve visually similar digit pairs such as 4/9 and 3/5, which is expected and consistent with human confusion patterns. This demonstrates that the remaining errors are not due to model weakness but rather the inherent ambiguity in certain handwritten samples.

4. Lessons and Experience Learned

Key Takeaways

- **CNNs are well-suited for image tasks:** Convolutional layers automatically learn spatial features (edges, curves, digit shapes) without manual feature engineering. This is a major advantage over traditional machine learning approaches that require hand-crafted features.
- **Data normalization matters:** Normalizing inputs to zero mean and unit variance helped the model train faster and more stably. Without normalization, gradient magnitudes can vary wildly across features, leading to slower convergence or training instability.
- **Simple architectures can be highly effective:** A two-layer CNN with only ~206K parameters achieved 98.84% test accuracy on MNIST. This demonstrates that more complex models are not always necessary - choosing the right architecture for the problem is more important than simply adding layers.
- **Reproducibility is important:** Setting a random seed ensured consistent results across runs, making it easier to compare experiments and debug issues during development. This is a best practice that should be followed in all machine learning projects.
- **PyTorch provides a clean workflow:** The combination of Dataset/DataLoader for data handling, nn.Module for model definition, and autograd for automatic differentiation made the end-to-end implementation straightforward and easy to understand.

Future Improvements

- Add dropout layers between fully connected layers to reduce overfitting on more complex datasets.
- Experiment with data augmentation techniques (rotation, scaling, elastic deformation) to improve model robustness.
- Try deeper architectures or add batch normalization for potentially higher accuracy.
- Implement a learning rate scheduler to fine-tune convergence in later epochs.
- Extend the project to more challenging datasets like Fashion-MNIST or CIFAR-10.

5. Acknowledgment

All of the ideas, concepts, analysis, and technical content presented in this report are entirely my own. I independently designed the CNN architecture, selected the hyperparameters, interpreted the results, and drew the conclusions discussed throughout this document. The project structure, implementation decisions, and all observations reflect my personal understanding of the material covered in the AI 100 course.

The coding and implementation part of this project was fully done by Claude (an AI assistant by Anthropic). This includes the training and evaluation code, utility functions, and the overall code structure. Additionally, after drafting the content of this report, I used Claude to help rewrite and polish the language for clarity, grammar, and overall readability. All ideas, technical decisions, analysis, and conclusions remain entirely my own.